

EXPERIMENT-10

ROLL NO: 240701058

NAME: ASHWIN C

USER INTERFACE AND DESIGN

AIM

The aim is to create data visualizations, such as bar charts, pie charts, and doughnut charts, for an inventory management system using JavaScript and Chart.js. The system displays stock quantity by product, category distribution, and reorder status with an interactive summary dashboard and detailed inventory table.

PROCEDURE

Step 1: Set Up Your HTML File

Create an HTML file to define the structure, including canvas elements where the charts will be rendered, and link the Chart.js library.

HTML (*index.html*):

```
<!DOCTYPE html>
<html lang="en">
<head>  <meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Inventory Data Visualization</title>
<link rel="stylesheet" href="style.css"> </head>
<body>  <main class="dashboard">
<header class="dashboard-header">
<div>
  <p class="eyebrow">Inventory Management System</p>
<h1>Inventory Data Visualization</h1>
</div>
  <p class="header-note">JavaScript charts for stock quantity, category share, and
reorder status.</p>
</header>
<section class="metrics-grid" aria-label="Inventory summary">
<article class="metric">
<span>Total Products</span>
<strong id="totalProducts">0</strong>
</article>
<article class="metric">
<span>Total Stock</span>
<strong id="totalStock">0</strong>
</article>
<article class="metric warning">
<span>Low Stock Items</span>
<strong id="lowStockItems">0</strong>
```

```

</article>
<article class="metric">
<span>Inventory Value</span>
  <strong id="inventoryValue">Rs. 0</strong>
</article>    </section>
<section class="chart-grid" aria-label="Inventory charts">
<article class="chart-panel wide">    <div class="panel-header">
<h2>Stock Quantity by Product</h2>
<p>Bar graph showing available units for each product.</p>
<div>    <div class="chart-wrap bar-chart">    <canvas
id="stockBarChart"></canvas>
  </div>    </article>
<article class="chart-panel">
<div class="panel-header">
<h2>Category Distribution</h2>
<p>Pie chart showing stock share across categories.</p>
  </div>
  <div class="chart-wrap pie-chart">
<canvas id="categoryPieChart"></canvas>
</div>    </article>
<article class="chart-panel">
<div class="panel-header">
<h2>Reorder Status</h2>
<p>Doughnut chart comparing safe stock and reorder items.</p>
</div>    <div class="chart-wrap pie-chart">    <canvas
id="reorderChart"></canvas>
</div>    </article>
</section>
<section class="table-panel" aria-labelledby="table-title">
<div class="panel-header">
<h2 id="table-title">Inventory Details</h2>
<p>Items below the reorder level are highlighted.</p>
</div>    <div class="table-scroll">
<table>
<thead>
  <tr>
<th>Product</th>
<th>Category</th>
<th>Stock</th>
<th>Reorder Level</th>
  <th>Unit Price</th>
<th>Status</th>
  </tr>
</thead>
<tbody id="inventoryTable"></tbody>
</table>
</div>
</section>
</main>
<script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.8/dist/chart.umd.min.js"></script>
<script src="script.js"></script> </body> </html>

```

Step 2: Create the CSS File for Styling

Create a CSS file to style the dashboard layout, metric cards, chart panels, and inventory table.

CSS (*style.css*):

```

* {
  box-sizing: border-box;

```

```
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: Arial, Helvetica, sans-serif;
  color: #172033;
  background: #f4f7fb;
}

.dashboard {
  width: min(1180px, 100%);
  margin: 0 auto;
  padding: 32px 18px 44px;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  gap: 24px;
  align-items: end;
  margin-bottom: 24px;
}

.eyebrow {
  margin: 0 0 10px;
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: #0f766e;
}

h1,
h2,
p {
  margin-top: 0;
}

h1 {
  margin-bottom: 0;
  font-size: clamp(32px, 5vw, 54px);
  line-height: 1.05;
  letter-spacing: 0;
}

h2 {
  margin-bottom: 6px;
  font-size: 20px;
}

.header-note,
.panel-header p {
  margin-bottom: 0;
  color: #64748b;
  line-height: 1.5;
}

.header-note {
  max-width: 360px;
}
```

```
.metrics-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
  margin-bottom: 18px;
}

.metric,
.chart-panel,
.table-panel {
  border: 1px solid #dbe4ef;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 12px 30px rgba(15, 23, 42, 0.06);
}

.metric {
  min-height: 112px;
  display: grid;
  align-content: center;
  gap: 8px;
  padding: 20px;
}

.metric span {
  color: #64748b;
  font-size: 14px;
  font-weight: 700;
}

.metric strong {
  font-size: 30px;
  color: #0f172a;
}

.metric.warning strong {
  color: #b45309;
}

.chart-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 18px;
  margin-bottom: 18px;
}

.chart-panel,
.table-panel {
  padding: 20px;
}

.chart-panel.wide {
  grid-column: 1 / -1;
}

.panel-header {
  margin-bottom: 18px;
}

.chart-wrap {
```

```
    position: relative;
    width: 100%;
}

.bar-chart {
    height: 360px;
}

.pie-chart {
    height: 320px;
}

.table-scroll {
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    min-width: 760px;
}

th,
td {
    padding: 14px 12px;
    border-bottom: 1px solid #e2e8f0;
    text-align: left;
    font-size: 14px;
}

th {
    color: #475569;
    background: #f8fafc;
}

tr.low-stock {
    background: #fff7ed;
}

.status {
    display: inline-block;
    min-width: 92px;
    border-radius: 999px;
    padding: 6px 10px;
    text-align: center;
    font-size: 12px;
    font-weight: 700;
}

.status.safe {
    color: #047857;
    background: #dcfce7;
}

.status.low {
    color: #b45309;
    background: #ffedd5;
}

@media (max-width: 860px) {
    .dashboard-header {
```

```

        display: grid;
        align-items: start;
    }

    .header-note {
        max-width: none;
    }

    .metrics-grid,
    .chart-grid {
        grid-template-columns: 1fr;
    }
}

@media (max-width: 560px) {
    .dashboard {
        padding: 22px 12px 32px;
    }

    .chart-panel,
    .table-panel,
    .metric {
        padding: 16px;
    }

    .bar-chart,
    .pie-chart {
        height: 280px;
    }
}

#ffedd5; }

```

Step 3: Create the JavaScript File for Charts

Create a JavaScript file to define the inventory datasets, calculate summary metrics, render the data table, and initialize the charts using the Chart.js constructor.

JavaScript (script.js):

```

const inventory = [
  { product: "Laptops", category: "Electronics", stock: 42, reorderLevel: 15,
    unitPrice: 54000 },
  { product: "Keyboards", category: "Electronics", stock: 95, reorderLevel: 30,
    unitPrice: 1200 },
  { product: "Office Chairs", category: "Furniture", stock: 18, reorderLevel: 20,
    unitPrice: 4500 },
  { product: "Desks", category: "Furniture", stock: 26, reorderLevel: 12, unitPrice:
    8200 },
  { product: "Printer Paper", category: "Stationery", stock: 140, reorderLevel: 50,
    unitPrice: 320 },
  { product: "Pens", category: "Stationery", stock: 260, reorderLevel: 100, unitPrice:
    12 },
  { product: "Monitors", category: "Electronics", stock: 21, reorderLevel: 18,
    unitPrice: 9800 },
  { product: "Storage Boxes", category: "Supplies", stock: 34, reorderLevel: 40,
    unitPrice: 180 }
];

const chartColors = [

```

```

"#2563eb",
"#0f766e",
"#f59e0b",
"#dc2626",
"#7c3aed",
"#0891b2"
];

function formatCurrency(value) {
  return `Rs. ${value.toLocaleString("en-IN")}`;
}

function groupStockByCategory() {
  return inventory.reduce((categories, item) => {
    categories[item.category] = (categories[item.category] || 0) + item.stock;
    return categories;
  }, {});
}

function updateSummaryCards() {
  const totalProducts = inventory.length;
  const totalStock = inventory.reduce((sum, item) => sum + item.stock, 0);
  const lowStockItems = inventory.filter((item) => item.stock <=
item.reorderLevel).length;
  const inventoryValue = inventory.reduce((sum, item) => sum + item.stock *
item.unitPrice, 0);

  document.getElementById("totalProducts").textContent = totalProducts;
  document.getElementById("totalStock").textContent = totalStock;
  document.getElementById("lowStockItems").textContent = lowStockItems;
  document.getElementById("inventoryValue").textContent =
formatCurrency(inventoryValue);
}

function renderInventoryTable() {
  const tableBody = document.getElementById("inventoryTable");

  tableBody.innerHTML = inventory
    .map((item) => {
      const isLowStock = item.stock <= item.reorderLevel;
      const statusClass = isLowStock ? "low" : "safe";
      const statusText = isLowStock ? "Reorder" : "In Stock";

      return `
        <tr class="${isLowStock ? "low-stock" : ""}">
          <td>${item.product}</td>
          <td>${item.category}</td>
          <td>${item.stock}</td>
          <td>${item.reorderLevel}</td>
          <td>${formatCurrency(item.unitPrice)}</td>
          <td><span class="status ${statusClass}">${statusText}</span></td>
        </tr>
      `;
    })
    .join("");
}

function createCharts() {
  const categoryStock = groupStockByCategory();
  const categoryLabels = Object.keys(categoryStock);
  const categoryValues = Object.values(categoryStock);

```

```

const lowStockCount = inventory.filter((item) => item.stock <=
item.reorderLevel).length;
const safeStockCount = inventory.length - lowStockCount;

new Chart(document.getElementById("stockBarChart"), {
  type: "bar",
  data: {
    labels: inventory.map((item) => item.product),
    datasets: [
      {
        label: "Available Stock",
        data: inventory.map((item) => item.stock),
        backgroundColor: "#2563eb",
        borderRadius: 6
      },
      {
        label: "Reorder Level",
        data: inventory.map((item) => item.reorderLevel),
        backgroundColor: "#f59e0b",
        borderRadius: 6
      }
    ]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    scales: {
      y: {
        beginAtZero: true,
        title: {
          display: true,
          text: "Units"
        }
      }
    },
    plugins: {
      legend: {
        position: "bottom"
      }
    }
  }
});

new Chart(document.getElementById("categoryPieChart"), {
  type: "pie",
  data: {
    labels: categoryLabels,
    datasets: [
      {
        label: "Stock by Category",
        data: categoryValues,
        backgroundColor: chartColors
      }
    ]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: {
        position: "bottom"
      }
    }
  }
});

```

```

    }
  }
}
});

new Chart(document.getElementById("reorderChart"), {
  type: "doughnut",
  data: {
    labels: ["In Stock", "Needs Reorder"],
    datasets: [
      {
        data: [safeStockCount, lowStockCount],
        backgroundColor: ["#0f766e", "#f59e0b"]
      }
    ]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    cutout: "62%",
    plugins: {
      legend: {
        position: "bottom"
      }
    }
  }
});
}

updateSummaryCards();
renderInventoryTable();
createCharts();

```

OUTPUT

The inventory data visualization dashboard renders the following components:

- Summary metric cards displaying Total Products (8), Total Stock (616 units), Low Stock Items (2), and total Inventory Value (Rs. 3,456,260).
- Bar chart comparing Available Stock vs Reorder Level across all 8 products.
- Pie chart showing category distribution: Electronics (158 units), Furniture (44 units), Stationery (400 units), and Supplies (34 units).
- Doughnut chart showing Reorder Status: 6 items In Stock and 2 items needing Reorder.
- Inventory details table listing all products with highlighted low-stock rows (Office Chairs and Storage Boxes).

Inventory Data Visualization

JavaScript charts for stock quantity, category share, and reorder status.

Total Products

8

Total Stock

636

Low Stock Items

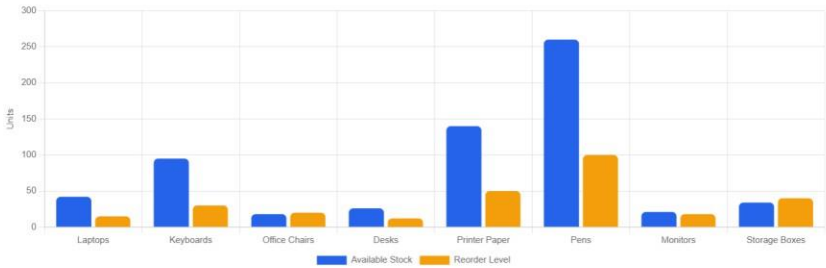
2

Inventory Value

Rs. 29,36,040

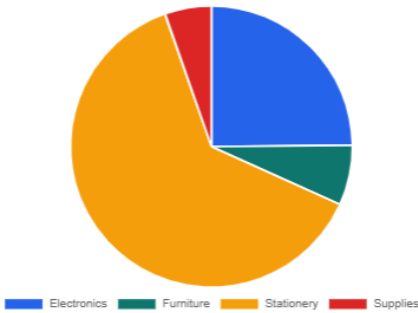
Stock Quantity by Product

Bar graph showing available units for each product.



Category Distribution

Pie chart showing stock share across categories.



Reorder Status

Doughnut chart comparing safe stock and reorder items.



Inventory Details

Items below the reorder level are highlighted.

Product	Category	Stock	Reorder Level	Unit Price	Status
Laptops	Electronics	42	15	Rs. 54,000	In Stock
Keyboards	Electronics	95	30	Rs. 1,200	In Stock
Office Chairs	Furniture	18	20	Rs. 4,500	Reorder
Desks	Furniture	26	12	Rs. 8,200	In Stock
Printer Paper	Stationery	140	50	Rs. 320	In Stock
Pens	Stationery	260	100	Rs. 12	In Stock
Monitors	Electronics	21	18	Rs. 9,800	In Stock
Storage Boxes	Supplies	34	40	Rs. 180	Reorder

RESULT

The data visualizations for inventory management were successfully created using JavaScript and Chart.js, providing a clear graphical representation of stock quantity by product, category-wise distribution, and reorder status. The dashboard enables quick identification of low-stock items and supports data-driven restocking decisions.